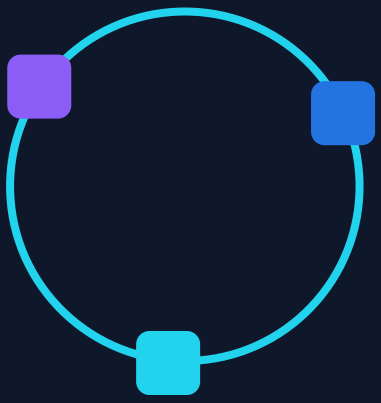
SOURCE-BASED ENGINEERING / BOOK GENERATION PACK



# Inside orch

## Building and Operating a Docker Orchestrator in Go

**Aleksandar Popovic**

alekpopovic.com

100 book-production prompts / 40 chapters

6 appendices / 8 progressive laboratories

Verified source history + one sample chapter

BLUEPRINT AND SAMPLE - THE FULL MANUSCRIPT IS PENDING

# A book grounded in executable behavior

This document accompanies the complete Codex generation pack for Inside orch: Building and Operating a Docker Orchestrator in Go, by Aleksandar Popovic. It gives the editorial plan, evidence boundaries, execution sequence and a finished source-based sample chapter.

The full 40-chapter manuscript is not included yet. The package contains the complete 100-prompt workflow that produces it, along with the locked source corpus and quality gates. Future PDF, EPUB and online editions are outputs of that workflow.

## The intended readers

| Reader | What the book should enable |
| --- | --- |
| Developer | Trace APIs and lifecycle transitions; reason about planner/reconciler invariants; test runtime side effects and integration seams. |
| DevOps engineer | Deploy an isolated lab, inspect actual runtime behavior, diagnose failures, validate recovery, and explain operational limits. |
| Architect | Evaluate state ownership, durability, concurrency, leases, fencing, isolation and capacity from mechanisms and evidence. |

## A distinctive teaching contract

Every chapter connects development intent, actual source changes, current code, an invariant, a failure window and observable evidence. Exact code excerpts have immutable links. Teaching extensions are visibly separate from upstream implementation. Exercises have worked answers.

The target is 85,000-110,000 meaningful words, excluding source listings and command output. This is an editorial target. The final page count will be measured after typesetting. Length cannot substitute for correct mechanisms, useful examples or tested procedures.

Every prompt executes from alekpopovic/alekpopovic.github.io repository root. The inspected Jekyll + Tailwind site has a native publisher, catalog, reader and completeness checks. Canonical book work lives under books/inside-orch; the pack preserves that contract and prepares local integration without publishing the live site.

# The immutable source edition

Source repository - Docker-first orchestration for small clusters.

Pinned HEAD: `cbfc94ade5b57e449d6b73bf1485b1c5b1647022`. This is a documentation update after the published 0.3.0 tag. Current code citations use this HEAD; historical explanations use the relevant historical commit.

| Evidence | Measured inventory |
|---|---|
| Tracked orch source files | 361 files with byte sizes and SHA-256 checksums |
| Native website contract | 20 selected source files, separately pinned at SHA 80801c8d3d97a4d8ff78e0fe69f3f902beda00c0 |
| Development prompts | 91 original prompts numbered 00-90 |
| Reachable commits | 105, with author/committer dates and exported diffs |
| Associated implementation/design commits | 66 distinct SHAs in the prompt crosswalk |
| Migrations | 16 up/down pairs |
| Book-production prompts | 100 full prompts numbered 000-099 |
| Source links checked in prompts | 508 immutable source/commit URLs |

## History requires careful attribution

The first 29 original prompt files were added after their June implementation. Their linked commits are retrospective topical associations, not proof of literal prompt execution at that date. Later changes batch several prompts in one commit. The original execution tracker is navigation evidence; it does not establish every requested guarantee.

| Release | Published UTC | Evidence |
|---|---|---|
| 0.1.0 | 2026-06-11T22:12:32Z | GitHub release |
| 0.3.0 | 2026-08-21T14:21:18Z | GitHub release |

Tag 0.3.0 points to `79d616acb679f237513b86eafe8c535983973579`; HEAD is `cbfc94ade5b57e449d6b73bf1485b1c5b1647022`. No 0.2.0 tag was present in the inspected repository. Release publication time and implementation commit time are different facts.

# What the book must explain honestly

The strongest material comes from the gap between a capable package and the executable that actually uses it. These are source-reviewed boundaries; runtime experiments remain tasks for the generated book.

| Topic | Source-based boundary |
| --- | --- |
| Durability | Default orch-server constructs MemoryService. PostgreSQL stores and migrations do not persist its live domain state. |
| Placement | MemoryService normally creates already assigned service tasks through its immediate reconciliation path. The standalone scheduler planner has a separate path. |
| Resource enforcement | The Docker adapter supports limits, but the active agent assignment/create path does not populate ContainerSpec.Resources. |
| Reconciliation | The standalone reconciler package is implemented; default startup instead relies on MemoryService reconciliation for normal services. |
| HA and autoscaling | Leadership and autoscaling packages exist; default server composition does not activate them as the claimed complete platform mechanisms. |
| Restart tests | The named rollout restart test reuses the same memory state. It demonstrates controller reconstruction, not whole-process durable recovery. |
| Partition recovery | Node timeout/replacement is different from fencing a still-live isolated workload. |
| Workload/platform scope | Local volume claims, partial GitOps applies, job/cron semantics, digest parsing and design-only features need precise boundaries. |

## Capability labels

Use runtime-integrated, implemented package, partial integration, design only, teaching extension and future work consistently. A package can be well-tested without being wired into a binary. Source inspection and executed validation are recorded separately.

# The complete writing sequence

| Prompt range | Concrete output |
|---|---|
| 000-019 | Research, source inventory, integration ledger, history, reader paths, lab environment and editorial foundation |
| 020-059 | All 40 chapters, examples, worked answers and chapter evidence ledgers |
| 060-065 | Six appendices with field/state/history/exercise/operator/glossary reference |
| 066-073 | Eight progressive laboratories and actual execution records |
| 074-085 | Code, source, lab, architecture, developer, operations, security and editorial reviews |
| 086-095 | Metadata, cover, PDF, EPUB, native Jekyll adapter, three posts, glossary and site checks |
| 096-099 | Release gate, independent review, archives/checksums and publication handoff |

## Execute from the website repository root

Extract the ZIP into alekpopovic/alekpopovic.github.io so books/inside-orch/ exists. Open the website repository root in Codex. Read root AGENTS.md, books/inside-orch/AGENTS.md, BOOK_SPEC.md and SITE_WORKSPACE.md. Execute books/inside-orch/prompts/000-workspace-and-baseline.md. Never replace the site root instructions or scripts. Original orch prompts 00-90 are evidence; do not rerun them as book-writing tasks.

```
python3 books/inside-orch/scripts/verify_pack.py
python3 books/inside-orch/scripts/next_prompt.py --show
```

The tracker begins pending for all 100 future book-production tasks. The provided source audits are seeds for verification. A done entry requires concrete evidence; blocked prerequisites and unrun experiments must stay visible.

## What has actually been checked

Orch and native-site source hashes, all 100 site-root execution bindings, prompt coverage, mapped commit objects, source paths and immutable links have been checked. Website builds have not been executed by this pack. The sample lab passes Bash syntax validation and was statically compared with flags, routes and configuration. Go is unavailable here, so the application build, Go tests, Docker/PostgreSQL labs and benchmarks have not been run. Expected sample observations are labeled accordingly.

# Forty chapters with distinct purposes

## I - Establish the System

### 01. Why Build a Container Orchestrator?

Define the small-cluster problem, desired-state architecture, and the project’s verified implementation boundary.

Lab / analysis: Draw the Atlas Lab system map and record the pinned source revision.

Original prompts: 00, 26. Writing task: 020.

### 02. A Repository Built for Reasoning

Read package boundaries, interfaces, injected dependencies, cancellation rules, and the repository’s engineering contract.

Lab / analysis: Create a read-only source index and run the prerequisite and build checks.

Original prompts: 00, 01, 21. Writing task: 021.

### 03. Boot the Lab and Deploy the First Service

Run the pinned local development path and inspect CLI-visible service, task, and node state.

Lab / analysis: Deploy the upstream http-api example, inspect events, scale, collect logs, then remove it.

Original prompts: 05, 21, 27, 28. Writing task: 022.

### 04. Domain Models and State Machines

Study service, task, node, deployment, and event types as explicit contracts.

Lab / analysis: Trace Atlas API lifecycle transitions with a state transition table and one rejected transition test.

Original prompts: 02, 29. Writing task: 023.

### 05. Service Specifications, Resources, and Ports

Read declarative service validation, resource quantities, labels, and host-port allocation constraints.

Lab / analysis: Create good and deliberately invalid Atlas manifests; explain replicas versus finite host-port capacity.

Original prompts: 17, 34, 43. Writing task: 024.

### 06. Follow One Request Through the Control Plane

Walk the default server assembly, in-memory control plane, controller loops, and runtime integration gaps.

Lab / analysis: Restart the local server in a disposable lab and contrast surviving Docker containers with lost control-plane state.

Original prompts: 04, 10, 25. Writing task: 025.

## II - Implement the Core

### 07. HTTP Contracts and Compatible APIs

Study middleware, validation, stable error responses, versioned routes, and the shared API contract.

Lab / analysis: Call the API with valid and invalid requests and add a book-side contract smoke test using the public client.

Original prompts: 04, 45, 61, 62. Writing task: 026.

## 08. A CLI That Operators Can Automate

Explore Cobra command composition, configuration precedence, JSON output, and deployment parsing.

Lab / analysis: Write an Atlas status script that uses current JSON fields and reports actionable failures.

Original prompts: 05, 43, 44. Writing task: 027.

## 09. Persistence, Transactions, and Optimistic Concurrency

Read PostgreSQL store interfaces and transactions while distinguishing them from the default in-memory server.

Lab / analysis: Run store integration tests against a disposable PostgreSQL database and examine a conflicting write.

Original prompts: 03, 24, 30. Writing task: 028.

## 10. A Deterministic Scheduler

Read placement order, eligibility checks, capacity accounting, and concurrent assignment safety.

Lab / analysis: Build a tiny fake-node placement experiment with valid, exhausted, and conflicting nodes.

Original prompts: 09, 17, 31. Writing task: 029.

## 11. Reconciliation as Repeated Convergence

Explain creation, scale-down, restart policy, soft deletion, and restart idempotency.

Lab / analysis: Apply the same desired Atlas replica count repeatedly in deterministic tests and verify no duplicate tasks.

Original prompts: 10, 24, 32. Writing task: 030.

## 12. Idempotent Docker Operations and Drift

Study the typed Docker adapter, managed labels, fake runtime, cleanup boundaries, and external container drift.

Lab / analysis: Repeat runtime operations through FakeRuntime and inject a missing or manually stopped managed container.

Original prompts: 06, 33, 41. Writing task: 031.

## 13. Agents, Heartbeats, and Task Reports

Trace registration, heartbeat, polling, local reconciliation, and status reporting.

Lab / analysis: Observe one Atlas worker agent and simulate dropped reports using deterministic fakes.

Original prompts: 07, 08, 19. Writing task: 032.

## 14. Health Checks That Respect Trust Boundaries

Read HTTP/TCP health checks, timeout behavior, status transitions, and SSRF controls.

Lab / analysis: Inject healthy, timeout, and forbidden-target probes for Atlas API without contacting metadata services.

Original prompts: 11, 53. Writing task: 033.

# III - Change and Recover

## 15. Events, Audits, and Notification Boundaries

Treat events as operator history, audit as an attribution record, and notifications as external effects.

Lab / analysis: Capture the Atlas deployment event timeline and send notifications to a local fake sink.

Original prompts: 13, 46, 80. Writing task: 034.

## 16. Log Streaming Without Losing Control

Trace server-to-agent log paths, Docker stream framing, cancellation, and bounded operator consumption.

Lab / analysis: Stream Atlas application logs, disconnect the client, and check cancellation through a local test.

Original prompts: 12. Writing task: 035.

## 17. Rolling Updates and Rollback Invariants

Explore service versions, deployment locks, max surge, max unavailable, and rollback state machines.

Lab / analysis: Roll Atlas API forward, inject an unhealthy target, and trace a rollback in fake and optional Docker lab modes.

Original prompts: 14, 15, 42. Writing task: 036.

## 18. Safe Deletion and Resource Cleanup

Follow deleting versus deleted services, task stop/remove behavior, finalization, and failures.

Lab / analysis: Delete Atlas API while injecting one remove failure; inspect repeated cleanup attempts.

Original prompts: 16, 25. Writing task: 037.

## 19. Offline Nodes, Draining, and Rescheduling

Read heartbeat expiry, node statuses, drain/uncordon operations, and workload rescheduling safety.

Lab / analysis: Pause a disposable Atlas agent and perform a deterministic drain/reschedule trace.

Original prompts: 39, 40. Writing task: 038.

## 20. Endpoint Discovery and Internal DNS

Study endpoint filtering, namespace names, host addresses and published ports, DNS TTL, and unhealthy endpoints.

Lab / analysis: Query Atlas endpoints and internal DNS under healthy and unhealthy fake task states.

Original prompts: 35, 78. Writing task: 039.

## 21. Ingress, Host Networking, and the Overlay Roadmap

Read Traefik configuration and current host-port traffic flow before evaluating future overlay/CNI design.

Lab / analysis: Route Atlas API with the upstream Traefik example and draw the current packet path.

Original prompts: 36, 77. Writing task: 040.

## 22. Autoscaling Decisions and Missing Integration

Read CPU recommendations, bounds, cooldown, rollout exclusion, and production provider gaps.

Lab / analysis: Drive Atlas replica recommendations with FakeMetricsProvider and inspect cooldown behavior.

Original prompts: 54, 55. Writing task: 041.

# IV - Enforce Boundaries

## 23. Authentication, Authorization, and Node Trust

Study operator JWT roles, agent enrollment tokens, route authorization, and node trust limits.

Lab / analysis: Run Atlas API authorization tests with viewer, operator, admin, and invalid agent credentials.

Original prompts: 18, 19, 51. Writing task: 042.

## 24. Encrypted Secrets and Configuration Redaction

Read secret envelope construction, namespace reference resolution, configuration precedence, and redacted output.

Lab / analysis: Deploy an Atlas placeholder-secret example and assert sample sensitive values are absent from printed configuration.

Original prompts: 37, 44. Writing task: 043.

## 25. Registry Credentials and Immutable Image Identity

Connect registry authentication, image references, tag policy, and digest pinning to rollout traceability.

Lab / analysis: Validate Atlas image references and use an explicitly labeled placeholder digest for non-executing examples.

Original prompts: 38, 67. Writing task: 044.

## 26. Security Contexts, Admission, and Policy

Read enforceable container security fields and policy/admission entry points.

Lab / analysis: Reject an unsafe Atlas manifest and inspect stable policy error details.

Original prompts: 52, 63, 64. Writing task: 045.

## 27. Namespaces and the Limits of Multi-Tenancy

Study namespace context, 404 isolation behavior, namespace-local uniqueness, role maps, and shared host risks.

Lab / analysis: Deploy two Atlas API services with the same name in different namespaces and verify invisible cross-namespace IDs.

Original prompts: 65. Writing task: 046.

## 28. Quotas and Usage Accounting

Read atomic quota checks, requested resource accounting, usage snapshots, and operational exports.

Lab / analysis: Exhaust an Atlas namespace quota and export a bounded usage report with known dimensions.

Original prompts: 66, 85. Writing task: 047.

## 29. Supply Chain Designs Without False Guarantees

Analyze image signing, SBOM and vulnerability scanning proposals against implemented image-policy boundaries.

Lab / analysis: Write an Atlas threat model and a book-side verification contract; mark every proposed adapter as illustrative.

Original prompts: 68, 69. Writing task: 048.

### 30. GitOps, Drift, and Controlled Reconciliation

Trace isolated git checkout, manifest parse/admission, recorded revision, warn/auto-revert drift, and prune ownership.

Lab / analysis: Use a local Atlas manifest repository, introduce manual drift, and demonstrate warn versus auto_revert.

Original prompts: 70, 71. Writing task: 049.

## V - Extend the Platform

### 31. Finite Jobs and Time-Based Work

Read Job attempts, retry limits, CronJob parsing, timezone semantics, concurrency policies, and missed runs.

Lab / analysis: Run a finite Atlas report job and drive CronJob fake time through overlap and missed-occurrence cases.

Original prompts: 72, 73, 74. Writing task: 050.

### 32. Local Volumes, Claims, and Workload Topology

Study local volume claims, scheduler topology, active writer locks, attachment lifecycle, and unsupported distributed storage.

Lab / analysis: Attach a disposable Atlas local volume and reject a second writer or wrong-node placement in tests.

Original prompts: 75, 76. Writing task: 051.

### 33. Extension Interfaces and a Plugin Roadmap

Read existing side-effect interfaces and design external RPC plugin lifecycle without inventing a plugin implementation.

Lab / analysis: Create an Atlas adapter contract exercise and label the external RPC protocol as a design proposal.

Original prompts: 79. Writing task: 052.

### 34. Leader Election and the Durable HA Boundary

Study lease interfaces, PostgreSQL advisory locks, leadership cancellation, and the default server integration gap.

Lab / analysis: Run deterministic competing-leader tests and draft a separate durable-server wiring ADR for Atlas.

Original prompts: 56, 57. Writing task: 053.

### 35. Upgrades, Migrations, Maintenance, and Retention

Read version compatibility, safe migration locks, down guards, maintenance windows, and bounded retention.

Lab / analysis: Perform a disposable Atlas migration-status and upgrade-preflight exercise, then dry-run retention.

Original prompts: 81, 82, 83, 84. Writing task: 054.

# VI - Prove and Operate

## 36. Fakes, Property Tests, Fuzzing, and Contracts

Use existing deterministic fakes, seeded scheduler properties, fuzz targets, and E2E tests as executable specifications.

Lab / analysis: Add book-side exercises for duplicate placement, parser panic resistance, and repeated convergence.

Original prompts: 22, 33, 86, 87. Writing task: 055.

## 37. Failure Injection and Recovery Evidence

Turn failure-mode hardening and chaos scenarios into bounded recovery investigations.

Lab / analysis: Run Atlas failure drills with fake runtime faults and optional local process interruption.

Original prompts: 25, 26, 59. Writing task: 056.

## 38. Metrics, Dashboards, and Actionable Signals

Trace Prometheus instruments, cardinality, alert rules, dashboard panels, and links to operator actions.

Lab / analysis: Inspect Atlas metrics and validate one real alert expression and dashboard query against emitted metric names.

Original prompts: 20, 46, 47, 48. Writing task: 057.

## 39. Performance and Scale Tests That Say What They Measure

Read fake-agent load tests, scheduler/API benchmarks, pprof controls, and large-service stress scenarios.

Lab / analysis: Run a bounded Atlas benchmark and create a measured-results template without fabricated data.

Original prompts: 58, 88, 89. Writing task: 058.

## 40. Release a System You Can Honestly Operate

Bring together reproducible release checks, deployment manifests, recovery drills, readiness limits, and an evidence-based production backlog.

Lab / analysis: Assemble Atlas evidence: tested lab, unresolved gaps, backup drill, compatibility preflight, and publication-ready case study.

Original prompts: 23, 27, 28, 49, 50, 60, 90. Writing task: 059.

# Appendices and progressive laboratories

The native reader has 47 sections: a separate preface first, then 40 numbered chapters and six appendices. Canonical selection follows books/inside-orch/manuscript/Book.txt. Chapter titles use Chapter N - Title; appendices use Appendix A - Title, matching the existing classifier.

### A. CLI, API, and Configuration Reference

Produce a source-derived command, route, field and configuration reference.

### B. Lifecycle States and Invariant Reference

Define lifecycle states, transitions, ownership and concurrency invariants.

### C. Commit Atlas and Original Prompt Crosswalk

Explain the 91 original prompts and real implementation associations across 105 commits.

### D. Worked Exercises and Design Solutions

Collect tested and source-justified answers keyed to chapters and labs.

### E. Operator Runbooks and Measurement Contracts

Provide observable triage, backup boundary, migration, maintenance and measurement procedures.

### F. Glossary, Bibliography, and Boundary Index

Define technical terms and primary references; link the capability/extension backlog.

## Eight laboratory stages

| Lab | Question to resolve |
|---|---|
| 01 | What survives a fresh default-server process, and what is lost? |
| 02 | What does the standalone planner/claim path guarantee under fit and concurrency conflicts? |
| 03 | How do repeated reconciliation and Docker adoption behave after drift or a lost report? |
| 04 | Which health/surge/availability conditions advance rollout and rollback? |
| 05 | Where do credentials, encrypted secrets and health probes cross trust boundaries? |
| 06 | What do namespace boundaries and quota decisions enforce in actual call paths? |
| 07 | How do GitOps partial failure and finite/time-based workloads converge? |
| 08 | What do migration/backup/load experiments prove, and what remains outside durable runtime recovery? |

# The quality gate is part of the book

Every future chapter needs source evidence, a distinct engineering problem, a specific failure window, a safe experiment and worked answers. Mechanical checks catch changed corpus bytes and missing artifacts; independent review checks the meaning. Neither a tracker nor a word count establishes correctness.

| Review | Required result |
| --- | --- |
| Source fidelity | Exact source listings regenerated; historical/current SHAs and renamed paths correct. |
| Execution | Actual commands, versions, UTC timestamps, exit codes and sanitized logs; all skipped external checks visible. |
| Claims | No false durability, fencing, atomicity, resource-enforcement or HA promise. |
| Readers | Developer, DevOps and architect reviews with concrete source-backed findings resolved. |
| Publication | All-page PDF render review; EPUB structure/reader checks; complete accessible online navigation and verified downloads. |
| Release | Measured metadata, chosen rights, reproducible sources, exact final hashes and a reviewable site patch. |

## Rights and publication

The author is Aleksandar Popovic. This pack has not selected a new public manuscript license or relabeled the source repository. No source license metadata was observed at the baseline. Rights, attribution and reused asset notices are reviewed explicitly before a public edition.

The pack targets scripts/book_catalog.py and scripts/build-books.py in the actual website. Final integration includes three original posts about different chapters, an inside-orch glossary and stable updates entry. Generated book-pages/inside-orch, assets/books/inside-orch and _data/books.json stay ignored. The native build checks completeness and actual download formats. Live publication remains a separate authorized action; no application or website repository was changed by this delivery.

# Sample chapter

The following is a finished source-based specimen of the intended writing standard. It belongs to the early runtime-boundary material. It is not a substitute for the planned 40-chapter manuscript. Its code excerpts are unchanged; its shell experiment is a labeled teaching addition with expected, unexecuted observations.

# From a Capable Package to a Restart-Safe Orchestrator

Aleksandar Popovic

An orchestrator is judged when its process disappears. Can another process recover the desired services, recognize existing containers, resume an update, and avoid creating competing work? A package can implement each of those ideas without the executable delivering their combined guarantee. The constructor graph decides which implementation actually runs.

This chapter reads orch at commit `cbfc94ade5b57e449d6b73bf1485b1c5b1647022`. All source links are pinned to that snapshot. It contains substantial scheduling, reconciliation, PostgreSQL, agent, and leadership code. Its default server nevertheless constructs an in-memory control plane. Understanding that distinction gives developers a precise integration target, gives operators realistic restart expectations, and gives architects a way to evaluate claims about durability.

The scope is specific: the normal `orch-server` startup path in this snapshot. Migration commands, standalone packages, future changes, and custom compositions can behave differently. We will trace the current path, run a disposable observation lab, and define the evidence required to support a stronger deployment claim.

## Start with the composition root

The composition root is the place where interfaces acquire concrete implementations. In orch it is the `run` function in cmd/orch-server/main.go. Read that function before drawing the production architecture.

Unchanged source excerpt - main.go, lines 146-148:

Print presentation wraps long lines; the companion Markdown and script preserve the copyable source.

```
controlPlane := controlplane.NewMemoryService(controlplane.WithSecretEnvelope(envelope),
     controlplane.WithClusterPolicy(cfg.ClusterPolicy),
    controlplane.WithRetentionConfig(cfg.Retention))
controlPlane.SetNotificationDispatcher(notifications.NewDispatcher(controlPlane, nil))
serverMetrics := metrics.NewServer()
```

The first line determines where desired services, nodes, tasks, deployments, secrets, and other domain records live. `MemoryService` owns maps guarded by a mutex. Those maps belong to this process. Encrypting a secret before storing it protects its representation, but does not make the map survive process termination.

Later startup code passes the same `controlPlane` into the GitOps controller, cron controller, scheduler, rollout controller, node monitor, and API handler. This is coherent dependency injection. It is also shared volatile state. The database URL does not replace that object. Startup

checks the database schema only when `ORCH_SCHEMA_CHECK` is true; migration subcommands are separate paths.

The operations and release hardening commit changed this entrypoint, while the jobs, storage, DNS, and notifications commit added further surfaces. Those links explain development history. Their subjects do not establish persistence: the current constructor remains the evidence for current behavior.

## Trace the path that creates a task

It is tempting to see `scheduler.New(controlPlane)` in startup and conclude that every service passes through the deterministic planner. Follow the write path before accepting that conclusion.

`CreateService` calls `reconcileServiceTasksLocked`. That method finds ready nodes, resolves volume constraints, and creates tasks already assigned to nodes. Its node selection contains this expression in internal/controlplane/controlplane.go:

Unchanged source excerpt - reconcileServiceTasksLocked, line 2292:

```
node := eligibleNodes[len(active)%len(eligibleNodes)]
```

The task literal below it sets `NodeID` and `ActualStatus: types.TaskAssigned`. The standalone scheduler lists pending tasks. A service task created this way has already crossed that boundary. The immediate memory reconciliation path therefore bypasses the standalone planner's resource and placement decisions. Jobs, which are created pending, provide a different path through the scheduler.

This distinction matters in a resource chapter. internal/scheduler/scheduler.go has meaningful logic: ready-node filtering, label constraints, CPU and memory requests, port reservations, volume affinity, deterministic scoring, and stable ordering. Those properties describe the planner. They do not automatically describe a service assigned elsewhere.

There is a second boundary at container creation. The Docker adapter accepts resource limits and converts them into Docker `Memory` and `NanoCPUs`. However, the agent's ensureTask method does not populate `ContainerSpec.Resources`, and the service assignment DTO does not carry a corresponding resource field. Thus this snapshot does not propagate ordinary service limits through the active agent path. Parsing a resource field, planning against requests, and enforcing a cgroup limit are three distinct obligations.

## A database implementation is one layer

The PostgreSQL store is real code, not merely a design document. It connects through pgx, implements transactions, stores core domain objects, and uses optimistic conditions on task mutations.

Unchanged SQL excerpt - AssignTask, line 919:

```
WHERE id = $1 AND updated_at = $5 AND actual_status = 'pending'
```

That condition prevents a stale claimant from assigning a task whose timestamp or state changed. The scheduler also groups assignment and its event through `store.WithTx` when the store supports transactions. These are useful building blocks for reliable persistence.

They solve bounded problems. A task-row condition does not atomically reserve all node capacity or all ports considered in an earlier planning snapshot. A transactional event append does not make the normal API use PostgreSQL. A migration for jobs does not prove the PostgreSQL store implements the same job behavior as `MemoryService`.

A durable composition needs a domain service that implements the API's control-plane contract using durable stores, including the later workload features. It also needs restart recovery across service versions, deployment state, credential records, and agent reports. Simply replacing the constructor with `NewPostgresStore` is not a complete solution: the store and control-plane service expose different responsibilities and interfaces.

## Leadership does not reconstruct state

The leader-election implementation commit introduced useful primitives. internal/leadership/leadership.go defines a lease with a `Done` channel and a wrapper that cancels controller work when leadership ends. The PostgreSQL elector acquires a session advisory lock on a dedicated pooled connection.

The default server does not wrap its controller loops with `RunWithLeadership`. Launching two default servers therefore creates two independent memory control planes. A load balancer cannot turn them into shared durable state.

Even after wiring the elector, lease cleanup deserves a real database test. In internal/leadership/postgres.go, cancellation closes the lease and releases its connection to the pool; `Release` returns early if the lease is already closed. Returning a pooled connection is different from explicitly ending the PostgreSQL session that owns an advisory lock. Test cancellation and failed unlocks before asserting those paths release leadership correctly. This is a source-derived review concern; this chapter does not report an observed lock leak.

State durability, leader coordination, and fencing are separate. A leader lock can limit controller writers while shared state is available. It does not restore missing service records. It does not by itself stop an old container on an isolated worker. Each guarantee needs a defined owner and failure test.

## Follow a server restart as an incident

Imagine one server and two agents serving an application. The operator restarts the server to apply a configuration change. Docker may keep the workload containers alive, but the new server has fresh control-plane maps. The desired service records and node credentials from the old process are gone.

The agents still hold old node IDs and credentials. Their heartbeat requests cannot be authorized against the previous records because those records no longer exist in the new `MemoryService`. The agent heartbeat retry loop retries rather than automatically rebuilding a durable identity. Workload continuity in Docker can therefore coexist with loss of orchestration control.

Do not turn this scenario into a promise that every container will immediately stop or that agents will automatically recover. Exact outcomes depend on which process restarts, how credentials are restored, and when the agent can retrieve assigned tasks. The important

invariant is narrower: this startup path has no persisted record from which the new server can reconstruct ownership.

An incident report should distinguish application reachability from control-plane recovery. Record whether the original endpoint serves, whether the service appears in the API, whether agent heartbeats succeed, and whether a rollout can resume. A successful HTTP request to the application answers only the first question.

Recovery also depends on identity. A managed Docker container carries labels identifying its task, service, node, and version. Those labels let the agent adopt a container after a lost status report. They cannot recreate a complete desired-state history by themselves. A label saying “version 3” does not contain the previous rollout strategy, the encrypted secret records, or the operator's desired replica count.

This is why container survival and orchestrator recovery must have separate acceptance criteria. In a durable implementation, task identity persists before the external Docker effect. After a crash, the agent can repeat creation against that same identity, and the adapter can find the previous container. If recovery generates a new task ID, label-based adoption no longer targets the original object; duplicate work becomes possible unless another reconciliation mechanism resolves ownership.

For a developer, the useful question is where identity first becomes durable. For an operator, it is how to tell whether the running container is still owned by the recovered control plane. For an architect, it is what happens when the database record and external runtime effect disagree. Answering those questions requires a recovery protocol, not just a startup order.

## Disposable lab: observe the state boundary

Teaching addition - the following shell script is written for this chapter, not copied from orch. Run it from a checkout at the pinned commit, on a development machine with Go 1.25 or newer, Bash with TCP redirections, and curl. It builds into a temporary directory and starts only its own loopback server. It needs no Docker, agent, or database. Use a free local port; the example uses 18080.

The script intentionally starts with a clean environment and disables the schema check. User JWT authentication remains empty for this local observation. The server must stay bound to `127.0.0.1`. The database address is deliberately unreachable and contains only disposable example credentials.

```
set -euo pipefail
book_tmp=$(mktemp -d)
book_server_pid=
book_port=18080

cleanup() {
  if [ -n "$book_server_pid" ]; then
    kill -TERM "$book_server_pid" 2>/dev/null || true
    wait "$book_server_pid" 2>/dev/null || true
  fi
  rm -rf "$book_tmp"
}
trap cleanup EXIT
trap 'exit 130' INT
trap 'exit 143' TERM
```

```
go build -o "$book_tmp/orch-server" ./cmd/orch-server

book_curl() {
  curl --noproxy '*' --connect-timeout 1 --max-time 2 -fsS "$@"
}

start_server() {
  if (exec 3<>"/dev/tcp/127.0.0.1/$book_port") 2>/dev/null; then
    printf 'Local port %s is in use; choose a free port.\n' "$book_port" >&2
    return 1
  fi
  env -i PATH="$PATH" ORCH_SCHEMA_CHECK=false \
    "$book_tmp/orch-server" \
    --addr="127.0.0.1:$book_port" \
    --database-url='postgres://lab:lab@127.0.0.1:1/lab?sslmode=disable' \
    --secret-key='disposable-book-lab-key' \
    >"$book_tmp/server.log" 2>&1 &
  book_server_pid=$!
  for attempt in {1..50}; do
    kill -0 "$book_server_pid" 2>/dev/null || {
      cat "$book_tmp/server.log"
      return 1
    }
    if book_curl "http://127.0.0.1:$book_port/readyz" >/dev/null 2>&1; then
      sleep 0.1
      if kill -0 "$book_server_pid" 2>/dev/null; then
        return 0
      fi
      cat "$book_tmp/server.log"
      return 1
    fi
    sleep 0.1
  done
  cat "$book_tmp/server.log"
  return 1
}

start_server
book_curl -X POST "http://127.0.0.1:$book_port/v1/namespaces" \
  -H 'Content-Type: application/json' \
  --data '{"name":"book-restart-lab"}'
book_curl "http://127.0.0.1:$book_port/v1/namespaces" \
  >"$book_tmp/before.json"

kill -TERM "$book_server_pid"
wait "$book_server_pid" || true
book_server_pid=
start_server
book_curl "http://127.0.0.1:$book_port/v1/namespaces" \
  >"$book_tmp/after.json"

cat "$book_tmp/before.json"
cat "$book_tmp/after.json"
```

Expected observations, not recorded execution results: the first response creates `book-restart-lab`; the first list contains it; after a fresh process starts, the second list lacks it and includes the initial default namespace. The server starts without connecting to the intentionally unavailable database because the normal path constructs memory state and the optional schema check is disabled.

If startup fails, inspect its log before interpreting output. If another service owns the port, choose a free one. Do not run the lab against a remote server or reuse a production database URL. No database reset command is required for this demonstration.

The endpoint called `readyz` returns a fixed ready response in this snapshot. Its success establishes that the HTTP handler is serving, not that durable state, database availability, controller leadership, or agent recovery has been verified.

## Turn observations into engineering acceptance criteria

A stronger restart claim needs a process-level test that creates meaningful state, terminates the server, starts a new process, and queries the same durable database. Include an in-progress rollout, service version history, and agent credentials. Keep the old containers alive so the new server must recognize them instead of hiding the problem by recreating everything.

Then test ambiguity. Terminate after a database write but before the API response. Terminate after Docker creates a container but before the status report succeeds. Repeat the operation after restart and count objects. Task identity is useful here: the adapter's CreateContainer implementation searches for an existing managed container by task ID before creating one. That supports adoption when the durable control plane still knows the task identity.

The chaos subtest named “server restarts during rollout” in internal/e2e/chaos_test.go constructs another rollout controller over the same memory service. It provides evidence for controller reconstruction with preserved state. It is not the process-level test just described.

Finally, make test prerequisites visible. PostgreSQL and Docker suites are opt-in and skip when their environment variables are absent. A green default `go test ./...` result cannot establish those external behaviors. Some database fixtures reset schema, so the appropriate test target is a disposable database dedicated to the lab.

## Exercises and answers

1. Why does setting `DATABASE_URL` fail to make the default server durable?

Answer: Configuration supplies a value; the composition root still constructs `MemoryService`. Persistence requires an implementation change in the domain/API path, with all relevant operations using durable storage and recovery tests.

2. Why can planner resource tests pass while an ordinary service exceeds node capacity?

Answer: The tested pure planner and the memory service's direct assigned-task path are different control flows. The latter does not call the former's fit checks. Trace the actual caller before transferring a package invariant to a deployment guarantee.

3. What would a two-server leadership test need to prove?

Answer: Both processes share durable state; only one controller callback is active per lock; losing the lease cancels work promptly; cancellation releases or ends the lock-owning session; a successor resumes without duplicate decisions. Independent memory maps cannot satisfy the shared-state condition.

4. Which missing link prevents Docker resource enforcement?

Answer: The active assignment/agent path does not deliver service limits into `ContainerSpec.Resources`. Adapter support is present, but the producer must populate the fields and a Docker inspection test must confirm the resulting HostConfig.

These limits define concrete integration work for this snapshot. The existing packages supply useful algorithms and boundaries. The next architectural task is to connect those boundaries so their individually tested behavior survives real process loss.

# The supplied package at a glance

| Path | Purpose |
|---|---|
| START_HERE.md | English site-root quick start and exact first task |
| SITE_WORKSPACE.md | Native Jekyll paths, ownership and publisher/check contract |
| ALL_100_PROMPTS.md | All full prompt texts in one file |
| prompts/ | 100 individually executable book-production tasks |
| editorial/ | Outline, history and technical seeds, claim/source policy and quality gates |
| evidence/ | Immutable orch snapshot/history plus 20 pinned native-site contract files |
| samples/ | Source-based sample chapter and Bash teaching lab |
| examples/source-excerpts/ | Six exact excerpt seeds with checksums and pinned line links |
| scripts/ | Corpus validation, excerpt extraction, tracker management and future manuscript structure checks |
| state/ | Machine-readable prompt index, all-pending tracker and pack verification result |
| publication/ | Consistent author/title/edition metadata |

All paths above are under books/inside-orch. Start prompt 000 from the website repository root, keep both source locks immutable, and retain real validation as the book grows. The final artifact should let readers inspect the code, reproduce the lesson and understand exactly where its guarantees stop.